SURVEILLANCE TRANSACTIONNELLE AML/CFT

INDUSTRIALISATION ET RESTITUTION

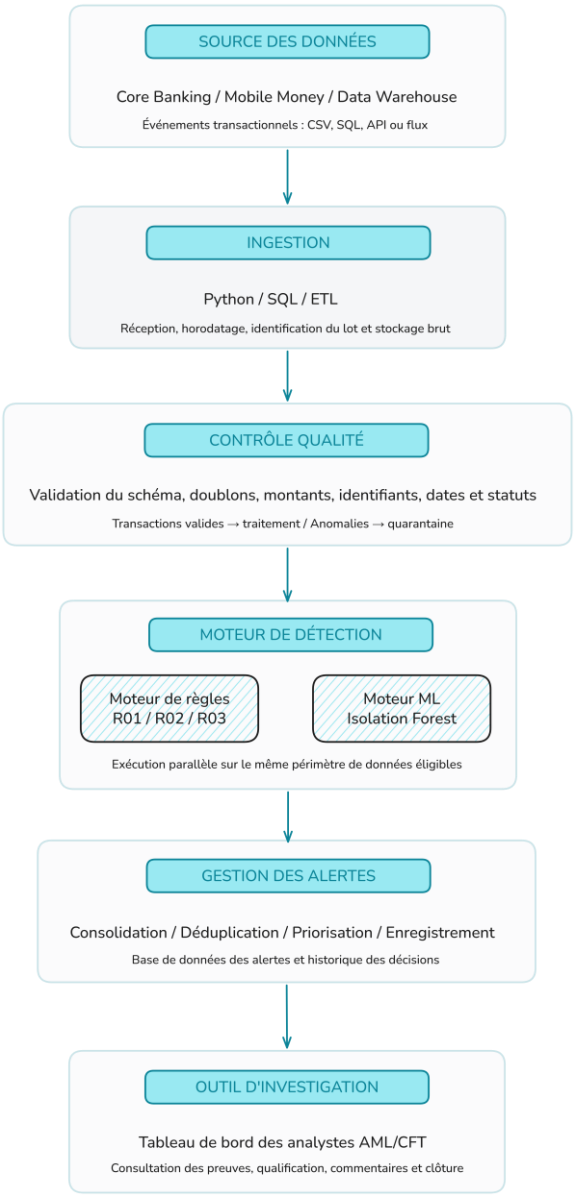
Objectif :

Construire une solution fiable, sécurisée, traçable, testable et évolutive.

1. Architecture technique cible

Architecture proposée

Je recommande une architecture modulaire avec un traitement batch quotidien dans notre premier périmètre.



Rôle des composants

Composant	Responsabilité	Prototype / cible
Source	Fournir les événements transactionnels	CSV / Data Warehouse, SQL ou API
Ingestion	Réception, lot, horodatage, conservation brute	Pandas / ETL / orchestrateur
Qualité	Contrôler schéma, IDs, montants, dates, doublons, statuts	Pandas / quarantaine
Règles	Détecter les scénarios métier connus	Python : R01, R02, R03
ML	Détecter les comportements atypiques	Scikit-learn : Isolation Forest
Gestion des alertes	Consolider, dédupliquer, stocker preuves et statuts	CSV prototype / PostgreSQL cible
Investigation	Consulter, qualifier, commenter, escalader et clôturer	Jupyter prototype / interface analyste
Orchestration	Planifier, relancer, surveiller et mesurer	Manuel prototype / **Airflow** ou équivalent

Une cible envisageable : Python, PostgreSQL, Scikit-learn, orchestrateur (Airflow) et interface d'investigation.

Services transverses : sécurité, habilitations, audit, versionnement, monitoring, configuration et reprise sur incident. Les choix définitifs dépendent de l'infrastructure, des SLA, de la sécurité et de la volumétrie.

2. Fiabilité opérationnelle

Reprise sur incident (fault tolerance)

Problème : si le pipeline s'arrête pendant R02, une relance complète peut recalculer les étapes réussies, perdre des résultats intermédiaires ou produire des doublons.

Solution : journal d'exécution par batch et par étape, avec **PENDING**, **RUNNING**, **SUCCESS** et **FAILED**. Au redémarrage, reprendre à la dernière étape validée. Sauvegarder durablement les résultats avant de marquer SUCCESS

```
CREATE TABLE pipeline_execution (
  batch_id VARCHAR(100) NOT NULL,
  step_name VARCHAR(50) NOT NULL,
  status VARCHAR(20) NOT NULL,
  started_at TIMESTAMP,
  completed_at TIMESTAMP,
  error_message TEXT,
  PRIMARY KEY (batch_id, step_name)
);
```

Scénario : R02 échoue. Ingestion, qualité et R01 restent SUCCESS. R02 est relancée, puis R03 et le ML selon les dépendances

Idempotence

Définition : rejouer le même événement ou traitement ne doit pas créer d'effet supplémentaire non maîtrisé. Protéger l'unicité des transactions et celle des alertes

```
CREATE TABLE transactions_processed (
  transaction_id VARCHAR(100) PRIMARY KEY,
  event_time TIMESTAMP NOT NULL,
  sender_id VARCHAR(100), receiver_id VARCHAR(100),
  amount_mga NUMERIC(18,2), status VARCHAR(20),
  batch_id VARCHAR(100),
  processed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

```
INSERT INTO transactions_processed (...)
VALUES (...)
ON CONFLICT (transaction_id) DO NOTHING;
```

Pour les alertes, calculer une clé stable sur rule_id, customer_id et les IDs de preuves triés/dédupliqués, puis imposer UNIQUE/PRIMARY KEY. Cela élimine les doublons exacts ; les fenêtres partiellement chevauchantes exigent en plus une logique de regroupement d'épisodes propre à chaque règle

Rapprochement des volumes (data reconciliation)

À chaque étape, mesurer : reçues, doublons, uniques, invalides, éligibles, traitées et non traitées. Tout écart doit être expliqué dans le rapport du batch

N_source = N_uniques + N_doublons

N_uniques = N_valides + N_invalides

N_valides = N_analysées + N_non_éligibles

Indicateur	Définition
source_count	Nombre de lignes reçues
duplicate_count	Lignes identifiées comme doublons selon la politique retenue
unique_count	Transactions uniques
invalid_count	Transactions uniques invalides
eligible_count	Transactions valides éligibles au moteur
processed_count	Transactions effectivement traitées
unprocessed_count	Éligibles non traitées ; nul ou expliqué
reconciliation_ok	Égalité entre éligibles et traitées

FAILED et **REVERSED** restent comptabilisés dans le rapprochement, mais ne sont pas traités par les règles définies sur **SUCCESS**. Aucune anomalie ne doit être supprimée silencieusement.

Surveillance des retards (late-arriving events)

event_time représente l'heure métier et **ingestion_time** l'heure de réception. Un événement tardif peut modifier une fenêtre déjà analysée. Il ne faut ni l'ignorer ni le compter comme une nouvelle transaction métier.

Solution : calculer le **retard** (ingestion_time – event_time), définir un *watermark* et une tolérance métier, puis traiter les événements tardifs par recalcul ciblé des fenêtres et clients concernés. L'idempotence et l'audit trail préviennent les doublons

Situation	Traitement
Événement reçu à temps	Batch normal
Événement après watermark	Traitement complémentaire
Événement déjà traité	Rejeu idempotent
Événement tardif modifiant une alerte	Réévaluation / mise à jour auditée

3. Sécurité, versionnement et traçabilité

Confidentialité

Protéger les données transactionnelles : chiffrement au repos et en transit (TLS), pseudonymisation, minimisation, sauvegardes protégées et exclusion des données sensibles des logs. Secrets et clés ne doivent jamais être dans le code ou notebook

Contrôle des accès

Rôle	Droits indicatifs
Administrateur	Gestion technique, utilisateurs et accès
Data Engineer	Pipelines et rapports qualité
AI Engineer	Features, modèles et métriques ML
Analyste AML/CFT	Consultation, investigation, qualification
Responsable Risques	Supervision, validation, reporting, escalade
Auditeur	Consultation des journaux et historiques, sans modification

Principes : moindre privilège, séparation des responsabilités, révocation rapide et journalisation des opérations sensibles. Les permissions sont imposées côté service et base, pas seulement dans le script.

Versionnement des règles et modèles

Pour chaque alerte, conserver les versions qui ont produit le résultat. Versionner les règles et paramètres séparément du code. Pour le ML, conserver algorithme, version, features_version, paramètres, date d'entraînement, dataset de référence et versions des bibliothèques

Métadonnée	Exemple
rule_version	R01 v1.0.0
model_version	IF_1.0.0
features_version	FEATURES_1.0
pipeline_version	PIPELINE_1.0.0
dataset_batch_id	BATCH_20261031
configuration_version	CONFIG_1.0.0
random_state / training_date	Graine et date pour reproductibilité

Git versionne le code et la configuration ; **MLflow** ou outil équivalent gère les versions, paramètres, artefacts et métriques ML. À sélectionner selon les contraintes de la banque

Audit trail des décisions

Séparer l'état courant de l'alerte de son historique append-only. Journaliser création, affectation, consultation sensible, investigation, transition de statut, commentaire, escalade et clôture : acteur, timestamp, ancien/nouveau statut et justification.

```
CREATE TABLE aml_alert_audit (
  audit_id BIGSERIAL PRIMARY KEY,
  alert_id VARCHAR(64) NOT NULL,
  action VARCHAR(50) NOT NULL,
  previous_status VARCHAR(30), new_status VARCHAR(30),
  performed_by VARCHAR(100) NOT NULL,
  justification TEXT,
  timestamp_utc TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

Une table SQL n'est pas automatiquement immuable : limiter **UPDATE/DELETE**, compte d'insertion dédié, centralisation sécurisée, sauvegardes et politique de rétention. L'alerte conserve également preuves, versions, paramètres et score éventuel.

Cycle de vie : NEW → ASSIGNED / IN_PROGRESS → INVESTIGATION → CLOSED (non pertinente ou justifiée) OU ESCALATED → décision finale.

Chaque transition est inscrite à l'audit trail.

4. Stratégie de tests

Tests unitaires

Règle	Cas	Attendu
R01	5 transactions admissibles, total > 4 000 000 MGA dans 60 min	Alerte
R01	4 transactions seulement	Pas d'alerte
R01	Total exactement 4 000 000 MGA	Pas d'alerte : seuil strict
R01	Transaction individuelle ≥ 1 000 000 MGA	Non admissible : règle exige < 1 000 000
R02	5 émetteurs distincts vers même bénéficiaire en 60 min	Alerte
R02	5 transactions mais 4 émetteurs distincts	Pas d'alerte
R03	Entrées puis sortie en 60 min, ratio > 80 % et ≤ 100 %	Alerte
R03	Ratio ≤ 80 % ou sortie > total entrées	Pas d'alerte

Tests d'intégration

Valider le pipeline complet : chargement → qualité → filtre SUCCESS → règles/ML → consolidation → export. Vérifier les colonnes obligatoires d'alerts.csv : alert_id, rule_id, customer_id, window_start, window_end, transaction_count, total_amount_mga, evidence_transaction_ids, explanation.

Tests de non-régression

Comparer la nouvelle sortie à un référentiel validé. Une différence volontaire suite à changement de seuil doit être documentée, approuvée et accompagnée d'une nouvelle version de référence.

Tests de performance

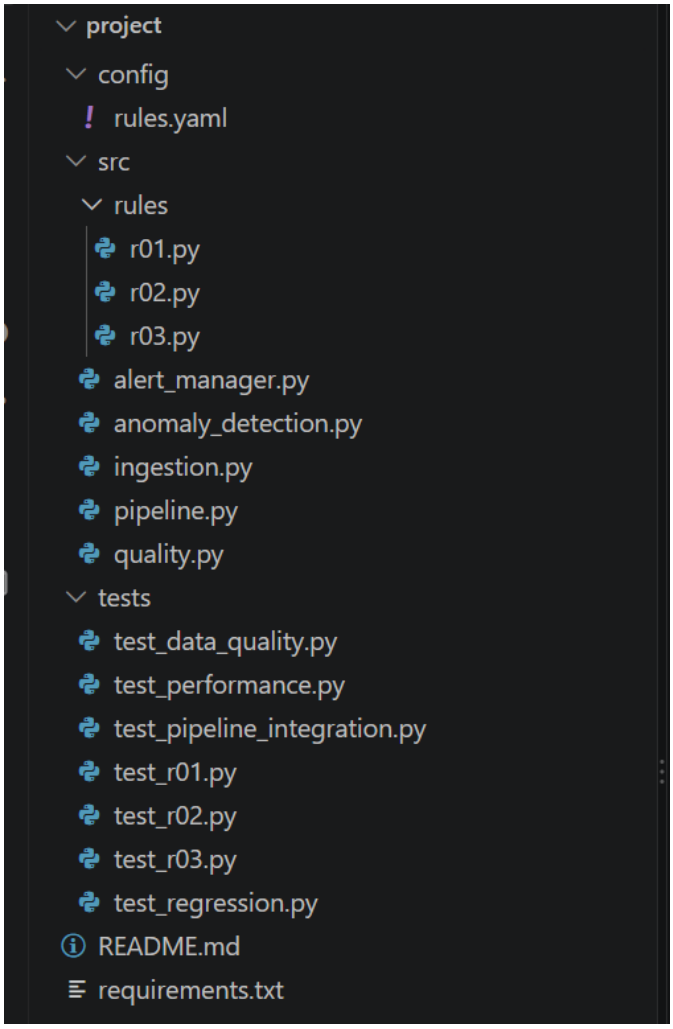
Mesurer durée d'ingestion, qualité, chaque règle, ML, consolidation, mémoire et nombre d'alertes. Tester par paliers (5 Mo, 50 Mo, 500 Mo, 1 Go par exemple). Ces volumes sont des scénarios de test, pas des SLA

Tests de robustesse

Cas injecté	Comportement attendu
ID transaction manquant	Anomalie qualité / quarantaine
Montant négatif ou invalide	Rejet contrôlé, non silencieux
Timestamp invalide	Erreur documentée et isolée
Doublon transaction_id	Déduplication et comptabilisation
Fichier vide / schéma incorrect	Arrêt contrôlé et explicite
FAILED / REVERSED	Exclus des règles SUCCESS, conservés au rapport
Transaction tardive	Traitement complémentaire selon watermark
Panne DB / interruption	Reprise contrôlée, pas de doublons

Automatisation

Organiser les tests dans tests/ et utiliser pytest. Exécuter les tests à chaque modification du code ou de configuration dans un pipeline CI/CD



5. Paramétrage dynamique

Principe

Séparer la logique de détection des paramètres métier. Une modification doit se faire dans un fichier de configuration, sans réécrire la fonction Python. Chaque changement est **validé**, **testé**, **approuvé** et **versionné**.

```
# config/rules.yaml
config_version: "1.0.0"
rules:
 R01:
  enabled: true
  min_transaction_count: 5
  max_individual_amount: 1000000
  min_total_amount: 4000000
  window_minutes: 60

 R02:
  enabled: true
  min_distinct_senders: 5
  window_minutes: 60

 R03:
  enabled: true
  min_outflow_ratio: 0.80
  max_outflow_ratio: 1.00
  window_minutes: 60
```

Chargement YAML

```
import yaml
from pathlib import Path

def load_config(config_path="config/rules.yaml"):
  path = Path(config_path)
  if not path.exists():
    raise FileNotFoundError(f"Configuration introuvable : {path}")
  with open(path, "r", encoding="utf-8") as f:
    return yaml.safe_load(f)

# pip install pyyaml
```

Le moteur lit **config['rules']['R01']** et utilise ses paramètres. La signature de la fonction doit être adaptée au code réel du notebook

Validation

Avant le batch, vérifier la présence des règles, le type booléen de enabled, la positivité des fenêtres/montants, les nombres minimaux et la cohérence des ratios. Une configuration invalide bloque le traitement.

```
def validate_config(config):
  rules = config.get("rules", {})
  for rule_id in ("R01", "R02", "R03"):
    if rule_id not in rules:
      raise ValueError(f"Règle manquante : {rule_id}")
    rule = rules[rule_id]
    if not isinstance(rule.get("enabled"), bool):
      raise ValueError(f"{rule_id}: enabled doit être booléen")
    if rule.get("window_minutes", 0) <= 0:
```

```
        raise ValueError(f"{rule_id}: fenêtre invalide")
    r01 = rules["R01"]
    if r01["min_transaction_count"] < 1:
        raise ValueError("R01: nombre minimal invalide")
    if r01["max_individual_amount"] <= 0 or r01["min_total_amount"] <= 0:
        raise ValueError("R01: montants invalides")
    if rules["R02"]["min_distinct_senders"] < 1:
        raise ValueError("R02: émetteurs invalides")
    r03 = rules["R03"]
    if not (0 < r03["min_outflow_ratio"] <= r03["max_outflow_ratio"] <= 1):
        raise ValueError("R03: ratios invalides")
    return True
```

Exemple de modification sans changement du code

Demande : augmenter le minimum R01 de 5 à 6 transactions et le montant total de 4 000 000 à 5 000 000 MGA. Modifier uniquement le YAML ; le code Python reste inchangé. Tester puis obtenir l'approbation métier avant activation.

```
R01:
  enabled: true
  min_transaction_count: 6
  max_individual_amount: 1000000
  min_total_amount: 5000000
  window_minutes: 60
```

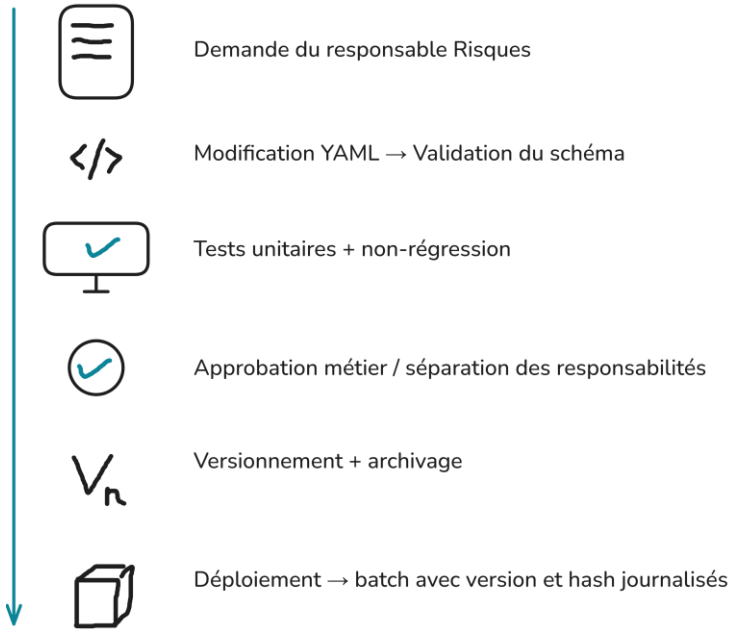
Version et empreinte

Chaque batch conserve config_version, pipeline_version, batch_id, date d'exécution et approbateur. Calculer une empreinte SHA-256 de la configuration normalisée pour vérifier son intégrité

```
import hashlib, json

def calculate_config_hash(config):
    payload = json.dumps(config, sort_keys=True).encode("utf-8")
    return hashlib.sha256(payload).hexdigest()
```

Workflow de changement



6. FAQ

Comment prouver que toutes les transactions pertinentes sont prises en compte?

Effectuer une réconciliation à chaque étape : reçues, doublons, uniques, invalides, éligibles, traitées et non traitées. Conserver un rapport par batch. Tout écart doit être nul ou expliqué par une catégorie documentée.

Comment empêcher qu'un rejeu ou une arrivée tardive crée des alertes incohérentes ?

Clé unique sur transaction_id, clé stable de déduplication d'alertes, journal de batch et checkpoints. Pour les événements tardifs : watermark, recalcul ciblé des fenêtres impactées et audit du recalcul. Le traitement doit rester idempotent

Comment réduire les faux positifs pour un commerçant à forte activité ?

Étudier le profil et l'activité habituelle : segment, fréquence, montants, saisonnalité, horaires, contreparties et historique. Envisager segmentation et features comportementales, puis ajuster les seuils après validation métier. Tester, versionner et surveiller les changements ; ne pas supprimer automatiquement les alertes sur le seul statut commerçant

Que faut-il journaliser pour justifier une alerte plusieurs mois après son émission ?

alert_id, rule_id ou model_version, configuration_version/hash, pipeline_version, batch_id, date de détection, fenêtre, transactions justificatives, features et score, explication, statut et historique des actions, identités, timestamps, commentaires et justification de clôture. Prévoir rétention et accès protégés

7. Conclusion

L'industrialisation ne consiste pas seulement à rendre le code exécutable. Elle garantit la fiabilité, la couverture, la reproductibilité, la sécurité, l'explicabilité et la maintenabilité. Le prototype évolue progressivement vers un pipeline batch supervisé et auditable, tandis que la décision finale d'investigation reste humaine